

1. Arquitectura General del Proyecto

El videojuego “El Último Bastión” se implementará siguiendo un modelo de arquitectura en capas y una máquina de estados para el flujo del juego. Esto permite separar la lógica de presentación, el control del flujo y la gestión de entidades, cumpliendo los principios de bajo acoplamiento y alta cohesión.

Funcionamiento general

- El motor principal es un bucle de juego (game loop) que controla el tiempo, procesa eventos, actualiza los estados y renderiza.
- El flujo del juego está gobernado por una máquina de estados que maneja las pantallas (menú, juego, pausa, derrota).
- Durante la fase de juego, un sistema de oleadas crea enemigos de forma progresiva. Los enemigos avanzan por un camino predefinido hacia el bastión. El jugador construye defensas en una cuadrícula usando recursos (oro, maná).
- Las defensas atacan automáticamente a los enemigos dentro de su rango. Los enemigos que alcanzan el bastión reducen su vida. Si la vida llega a 0, se activa la pantalla de derrota.

Módulos principales e interacción

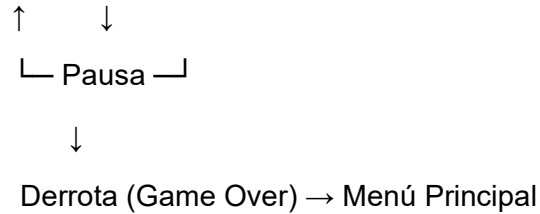
- **Core**: bucle principal, máquina de estados, configuración global.
- **Entities**: enemigos, defensas, proyectiles y el propio bastión.
- **Managers**: gestión de oleadas, recursos, colisiones, entrada del jugador.
- **Map**: cuadrícula de construcción, camino de enemigos, lógica de posiciones válidas.
- **UI**: renderizado de menús, HUD (vida, recursos, oleada actual), botones.
- **Utilities**: carga de assets, temporizadores, vectores matemáticos.

La comunicación se realiza mediante referencias a los managers a través del estado de juego actual (p.ej., `GameplayState` contiene instancias de `WaveManager`, `ResourceManager`, etc.). Los sprites se organizan en grupos de Pygame para el renderizado y la detección de colisiones.

Flujo del videojuego

...

Inicio → Menú Principal → Jugar → Juego (GameplayState)



...

2. Estructura de Carpetas Recomendada

...

el-ultimo-bastion/

```

├── main.py          # Punto de entrada, inicializa Pygame y arranca el juego
├── config.py        # Constantes globales (resolución, FPS, rutas, balances)
├── src/
│   ├── core/
│   │   ├── game.py    # Clase principal Game (bucle, estados)
│   │   └── states.py   # Estados de la máquina (MenuState, GameplayState, etc.)
│   ├── entities/
│   │   ├── enemy.py    # Clase base Enemy y subclases (Saqueador, Arquero,
│   │   │               Forzudo)
│   │   ├── defense.py  # Clase base Defense y subclases (TorreArquera, Muro,
│   │   │               Balista)
│   │   ├── projectile.py # proyectiles disparados por defensas
│   │   └── bastion.py   # El bastión (vida, posición)
│   └── managers/
│       ├── wave_manager.py # Control de oleadas, generación de enemigos
│       ├── resource_manager.py # Gestión de oro y maná, validación de gastos
│       ├── input_manager.py # Procesamiento de eventos de teclado y ratón
│       └── collision_manager.py # Lógica de colisiones entre proyectiles y enemigos
└── map/
  
```

- | | └─ grid.py # Cuadrícula para construir defensas
- | | └─ path.py # Definición del camino (waypoints)
- | | └─ tile.py # Representación de cada celda del mapa
- | └─ ui/
- | | └─ button.py # Botón genérico para menús
- | | └─ hud.py # Interfaz durante el juego (vida, recursos, oleada)
- | | └─ menu.py # Pantalla de menú principal
- | | └─ pause_menu.py # Pantalla de pausa
- | | └─ game_over.py # Pantalla de derrota
- | └─ utils/
- | └─ asset_loader.py # Carga de imágenes, sonidos, fuentes
- | └─ timer.py # Temporizador para cooldowns y oleadas
- | └─ vector.py # Clase Vector2D para cálculos matemáticos
- └─ assets/
- | └─ graphics/
- | | └─ enemies/ # Sprites de goblins
- | | └─ defenses/ # Sprites de torres, muros, balista
- | | └─ tiles/ # Imágenes del terreno
- | | └─ ui/ # Botones, fondos de menú
- | | └─ effects/ # proyectiles, explosiones (futuro)
- | └─ sounds/
- | | └─ music/
- | | └─ sfx/
- | └─ fonts/ # Fuentes bitmap
- └─ docs/ # Documentación técnica y de diseño
- └─ tests/ # Pruebas unitarias (p.ej., con pytest)
- └─ data/ # Archivos de definición de niveles, oleadas, balances
- └─ levels/
- └─ level_01.json

...

3. Módulos del Sistema

Módulo	Función principal
core	Bucle de juego (control del tiempo, actualización, renderizado) y máquina de estados. Coordina el cambio entre pantallas.
entities/enemy	Define los distintos tipos de enemigos, su comportamiento (movimiento por camino), atributos de vida/daño/velocidad y su representación visual.
entities/defense	Torres, muros y balista. Cada una tiene lógica de ataque (automático), cooldown, alcance y coste de recursos.
entities/bastion	Representa el corazón del bastión. Almacena los puntos de vida y detecta cuándo un enemigo llega para reducir la vida.
managers/wave	Controla la generación de oleadas: número de enemigos, tipos, intervalos. Incrementa la dificultad según la oleada actual.
managers/resource	Administra las cantidades de oro y maná, permite añadir/gastar recursos y notifica cuando no hay suficientes para construir.
managers/input	Captura eventos de Pygame (ratón, teclado) y los traduce a acciones del juego (seleccionar defensa, colocar, pausar).
managers/collision	Comprueba colisiones entre proyectiles y enemigos, entre enemigos y el bastión. Opcionalmente, entre enemigos y defensas si procede.
map/grid	Matriz de celdas donde se construyen defensas. Define qué posiciones son válidas (no bloqueadas por camino ni por otras construcciones).
map/path	Almacena una lista de waypoints que los enemigos siguen. Proporciona métodos para obtener el siguiente punto y la dirección.
ui/hud	Muestra en pantalla la vida del bastión, el oro, el maná y el número de oleada.
ui/menu	Pantalla de menú principal con opciones "Jugar" y "Salir".
ui/pause_menu	Menú de pausa que congela el juego y permite reanudar o salir.

| ****ui/game_over**** | Pantalla que se muestra al perder, con opción de volver al menú.
|

| ****utils/asset_loader**** | Carga y almacena en caché todas las imágenes y sonidos para evitar cargas repetidas. |

| ****utils/timer**** | Temporizador reutilizable para cooldowns de disparo y para los intervalos entre oleadas. |

| ****utils/vector**** | Clase auxiliar para operaciones con vectores (cálculo de dirección, distancia, normalización). |

4. Diseño de Clases

4.1 Clase `Game` (core/game.py)

- ****Responsabilidad****: Inicializar Pygame, administrar el bucle principal y delegar en el estado activo.
- ****Atributos****: `screen`, `clock`, `running`, `state` (objeto de estado actual).
- ****Métodos****: `run()`, `change\_state(new\_state)`, `handle\_events()`, `update(dt)`, `render()`.
- ****Relaciones****: Contiene una instancia de `State` (polimorfismo). Los estados concretos heredan de una clase base `State`.

4.2 Estados (core/states.py)

- ****Clase base `State`****: define `handle\_events()`, `update(dt)`, `render(screen)`.
- ****`MenuState`****: muestra fondo y botones. Crea `GameplayState` al pulsar "Jugar".
- ****`GameplayState`****: contiene la lógica principal del juego, instancia los managers y entidades.
- ****`PauseState`****: guarda referencia al `GameplayState` para reanudarlo.
- ****`GameOverState`****: recibe el motivo de derrota, muestra mensaje y permite volver al menú.

4.3 Clase `Enemy` (entities/enemy.py)

- ****Responsabilidad****: Moverse por el camino, recibir daño, causar daño al bastión.

- **Atributos**: ``health``, ``max_health``, ``speed``, ``damage``, ``position`` (Vector2), ``current_waypoint``, ``image``, ``rect``.
- **Métodos**: ``update(dt)``, ``take_damage(amount)``, ``reached_end()``, ``move_towards(target)``.
- **Subclases**:
 - ``GoblinSaqueador``: vida baja, velocidad media, daño normal.
 - ``GoblinArquero``: ataque a distancia (puede lanzar un proyectil o simplemente ser una variante que ataca torres; según especificación inicial solo “ataque a distancia” sin detalle, se asume que ataca al bastión a distancia). Vida media, velocidad media.
 - ``GoblinForzudo``: alta vida, daño elevado, velocidad baja.
- **Relaciones**: Conoce el ``Path`` (obtenido del mapa). Es parte de un grupo de sprites de Pygame para colisión y renderizado.

4.4 Clase ``Defense`` (entities/defense.py)

- **Responsabilidad**: Detectar enemigos en rango y disparar automáticamente. Muro solo bloquea.
- **Atributos**: ``cost_gold``, ``cost_mana``, ``range``, ``damage``, ``cooldown``, ``attack_timer`` (Timer), ``position`` (grid), ``image``.
- **Métodos**: ``update(dt, enemy_group)``, ``attack(enemy)``, ``can_afford(resource_manager)``.
- **Subclases**:
 - ``TorreArquera``: daño medio, rango medio, solo cuesta oro.
 - ``Balista``: alto daño, largo alcance, costo elevado de oro y algo de maná.
 - ``MuroDefensivo``: no ataca, bloquea el paso de enemigos (los enemigos no pueden atravesar, deben rodear o destruir el muro, aunque en la versión inicial simplemente detiene a los enemigos adyacentes; se simplifica como un obstáculo). Vida propia alta.
- **Relaciones**: Se coloca en un objeto ``Grid``. Interactúa con ``Enemy`` a través del ``CollisionManager`` y el grupo de enemigos.

4.5 Clase ``Projectile`` (entities/projectile.py)

- **Responsabilidad**: Moverse hacia un enemigo objetivo, infligir daño al impactar.
- **Atributos**: ``speed``, ``damage``, ``target``, ``position``, ``image``.
- **Métodos**: ``update(dt)``, ``has_hit_target()``.

- **Relaciones**: Creado por `Defense` al disparar. El `CollisionManager` verifica impacto.

4.6 Clase `Bastion` (entities/bastion.py)

- **Responsabilidad**: Almacenar la vida del bastión y recibir daño.
- **Atributos**: `health`, `max\_health`, `position`, `image`.
- **Métodos**: `take\_damage(amount)`, `is\_destroyed()`.
- **Relaciones**: Referenciado por `GameplayState` y por `CollisionManager` para detectar enemigos que llegaron.

4.7 Managers

Clase	Responsabilidad principal	
Atributos clave	Métodos importantes	
----- ----- -----		
----- ----- -----		
`WaveManager`	Generar oleadas, controlar tiempos entre oleadas y entre enemigos	
	`current_wave`, `enemies_to_spawn`, `timer`, `wave_data`	
	`start_next_wave()`, `update(dt)`, `spawn_enemy()`	
`ResourceManager`	Mantener el oro y maná del jugador, validar compras	
`gold`, `mana`	`add_gold(n)`, `spend(cost_gold, cost_mana)`,	
`can_afford(...)`		
`InputManager`	Mapear eventos a acciones del juego (click en cuadrícula, selección de defensa, pausa)	
	Referencia a `Grid` y `GameplayState`	
	`handle_event(event)`,	
	`get_clicked_cell()`	
`CollisionManager`	Detectar colisiones proyectil-enemigo, enemigo-bastión, y eventualmente enemigo-muro	
	`enemy_group`, `projectile_group`, `bastion`	
	`check_collisions()`	

4.8 Mapa

- **Grid**: matriz bidimensional de `Tile`. Método `place\_defense(pos, defense)`, `is\_occupied(pos)`, `is\_valid\_build\_position(pos)`.
- **Path**: lista de `Vector2` waypoints. Método `get\_next\_waypoint(current\_index)`.

- `Tile`: contiene tipo (camino, construible, no construible) y referencia a la defensa construida (o `None`).

4.9 UI

- `Button`: rectángulo, texto, callback. Maneja eventos de click.
- `HUD`: se actualiza cada frame con los datos de `GameplayState` (vida, oro, maná, oleada).
- `Menu`, `PauseMenu`, `GameOver`: heredan de un contenedor de UI básico (pueden ser simplemente funciones que dibujan usando `Button`).

5. Diagrama Conceptual (Flujo General del Sistema)

...

main.py

└─▶ `Game.run()`

|

▼

MenuState —(Jugar)—▶ GameplayState

|

└─ WaveManager —▶ genera —▶ Enemy (Goblin...)

|

└─ ResourceManager |

|

└─ InputManager Path (waypoints)

|

└─ └─▶ selecciona/coloca Defense (Torre, Muro, Balista)

|

▼

| Grid (celdas válidas)

|


```

└─ CollisionManager
  |
  └─ Proyectil vs Enemy → Enemy.take_damage()
    └─ Enemy vs Bastión → Bastion.take_damage()
      |
      └─ UI/HUD ← actualiza vida, oro, oleada
        |
        (vida ≤ 0) → GameOverState

```

...

6. Escalabilidad

La arquitectura propuesta permite extender el juego de manera natural gracias al polimorfismo y la separación de responsabilidades.

Nuevos enemigos

- Basta crear una nueva subclase de `Enemy` (por ejemplo `OrcoHechicero`) con sus propios atributos y, si es necesario, sobrescribir `update()` para comportamientos especiales (regeneración, invocación de esbirros). Se añade su sprite y se registra en los datos de oleada sin modificar el código central.

Nuevas torres/defensas

- Similar a enemigos: nueva subclase de `Defense` con distintos valores de rango, daño, cooldown y costes. El sistema de construcción es genérico (solo depende de la interfaz de `Defense`), por lo que la cuadrícula y los recursos funcionan sin cambios.

Nuevos mapas

- La clase `Path` y la definición de la `Grid` se cargan desde archivos externos (p.ej., JSON en `data/levels/`). Cada nivel puede especificar su propio camino (lista de waypoints) y la distribución de celdas construibles. El `GameplayState` recibe el identificador del nivel y usa un `LevelLoader` (o similar) para instanciar el mapa.

Nuevos modos de juego

- La máquina de estados (`core/states.py`) permite añadir nuevos estados sin afectar a los existentes. Un modo "Supervivencia infinita" podría ser un `EndlessState` que hereda de `GameplayState` y modifica la lógica de oleadas. También se pueden añadir transiciones a estados de victoria (si se implementan niveles con objetivo fijo).

Mejoras futuras

- ****Árbol de mejoras****: cada defensa puede tener un método `upgrade()` que incrementa sus atributos y cambia su sprite. Los recursos y la interfaz solo necesitan una comprobación adicional.

- ****Efectos visuales/partículas****: se puede añadir un sistema de partículas en `utils/` y un `EffectManager` sin impactar en las entidades principales.
- ****Sonido posicional y música****: el `AssetLoader` ya centraliza los recursos; se puede integrar un `AudioManager` que maneje canales.
- ****Guardado/carga de partida****: serializando el estado de los managers y la cuadrícula; la arquitectura basada en datos facilita esta tarea.
- ****Multijugador****: se requeriría un rediseño parcial, pero la separación entre lógica y presentación (estado del juego en `GameplayState` y render en la vista) es un primer paso hacia una arquitectura cliente-servidor.

7. Riesgos Técnicos

Riesgo	Probabilidad	Impacto	Mitigación
Rendimiento con muchos enemigos/proyectiles	Media	Alto	Usar grupos de sprites eficientes, limitar el número de enemigos simultáneos, optimizar colisiones (solo verificar enemigos activos).
Lógica de colisiones compleja (muros, enemigos, proyectiles)	Media	Medio	Implementar un `CollisionManager` centralizado con comprobaciones por capas. Dividir el espacio en sectores si es necesario.
Dificultad para balancear recursos/oleadas	Alta	Bajo	Externalizar los parámetros de balance a archivos JSON (`data/balance.json`) para ajustes sin recompilar.
Acoplamiento entre UI y lógica del juego	Media	Medio	Seguir el patrón observador o pasar referencias explícitas solo a través del estado de juego. El HUD consulta los managers pero no los modifica.
Carga y gestión de assets ineficiente	Baja	Bajo	Centralizar en `AssetLoader` con un diccionario caché. Cargar todos los recursos al inicio o bajo demanda con una pantalla de carga.
Manejo de eventos en tiempo real (click rápido)	Media	Bajo	El `InputManager` debe filtrar eventos repetidos (p.ej., mantener pulsado para construir). Usar temporizadores de cooldown en la construcción.
Pygame no soporta nativamente escalado fluido	Baja	Medio	Definir una resolución base fija y usar `pygame.transform.scale` o `pygame.display.set_mode` con banderas apropiadas. Planificar para futuros cambios de resolución.

8. Recomendaciones Profesionales

- ****Separación estricta de lógica y presentación****: El estado del juego (``GameState`` y sus managers) no debe tener referencias a elementos gráficos de Pygame (excepto grupos de sprites, que son un puente). Así se puede probar la lógica sin inicializar una ventana gráfica.
- ****Uso de datos externos para balance****: Crear un archivo ``config.json`` o varios en ``data/`` con las estadísticas de enemigos, defensas, costes y parámetros de oleadas. Esto facilita iterar sobre el balance y permite a diseñadores (o futuras IAs) modificar el juego sin tocar código.
- ****Patrón de estados para pantallas****: Implementar una máquina de estados limpia (como la descrita) evita “código espagueti” con banderas globales y hace que añadir nuevas pantallas (victoria, opciones, créditos) sea trivial.
- ****Pruebas unitarias desde el inicio****: Utilizar ``pytest`` para validar la lógica de los managers (oleadas, recursos) y de las entidades (movimiento, daño). Al ser un proyecto de investigación comparativa, la calidad del software debe ser medible; unas buenas pruebas incrementan la confianza en el código generado.
- ****Control de versiones con Git y commits atómicos****: Mantener un historial limpio facilita entender la evolución del proyecto y aísla cambios de cada IA si se generan variantes.
- ****Documentación en código (docstrings)****: Cada clase y método público debe describir su propósito, parámetros y valores de retorno. Esto ayuda a cualquier evaluador y a la propia IA que pudiera analizar el código más tarde.
- ****Gestión de recursos mediante ``pygame.sprite.Group``****: Aprovechar los grupos de Pygame para renderizado y colisiones (``pygame.sprite.groupcollide``) reduce el código manual y mejora el rendimiento.
- ****Delta time (dt) en todas las actualizaciones****: Para que el movimiento y los temporizadores sean independientes de los FPS. Todos los métodos ``update(dt)`` reciben el tiempo transcurrido desde el último frame en segundos.
- ****No sobrecargar el bucle principal****: El método ``GameState.update()`` delegará en los managers y estos en las entidades. Evitar lógica condicional anidada; preferir polimorfismo y delegación.
- ****Prototipado rápido del menú y la UI****: Aunque el foco está en la jugabilidad, una interfaz funcional desde el principio permite probar el flujo completo. Usar fuentes bitmap o ``pygame.font.SysFont`` para texto sencillo, y luego mejorar con fuentes personalizadas.
- ****Considerar un sistema de eventos interno****: Para comunicación entre módulos sin dependencias rígidas (por ejemplo, un enemigo muere → suelta oro). Un sencillo bus de eventos (una cola de mensajes) desacopla aún más los componentes, aunque para la escala inicial puede no ser necesario; dejarlo documentado como posible mejora.